Stronger Encoders and Structure-Constrained Decoding for DiCo-NLI (Track 1)

Muhammad Muneeb Baig, Syed Kamran Ali Shah, Kashaf Ali Haroon
MS Data Science, SEECS, NUST
CS-878 Deep Learning, Assignment 2

November 2026

Abstract

In Assignment 1 (A1), a fine-tuned RoBERTa-base reached 0.765 weighted F1 (W-F1) and 0.739 HardCons on the DiCo-NLI English development set, with most remaining errors on the NEG boundary rather than on direction. Here we propose and test four changes: a stronger encoder (DeBERTa-v3-base), a reversal-consistency training loss, NEG class weighting, and inference-time decoding rules that use the pair structure of the data. Over three seeds, DeBERTa-v3 improves argmax W-F1 from 0.758 to 0.821 and HardCons from 0.759 to 0.839. Symmetric decoding adds 2–6 HardCons points at no training cost. A property of the released data (rows with a reversed partner are never NEG, rows without one always are) gives the largest gain, to 0.925 W-F1 and 0.911 HardCons, but it is dataset-specific. The consistency loss shows no robust benefit. A three-seed ensemble scores 0.936 W-F1 and 0.924 HardCons, confirmed by a development-phase Codabench submission.

1 Introduction

DiCo-NLI (SemEval-2027 Task 2) asks a classifier to label an ordered phrase pair (t_1, t_2) as EQ, FWD, BWD or NEG, and scores both classification and directional coherence (DiCo-NLI Organizers, 2026). A coherent model satisfies $f(t_2, t_1) = \mathrm{rev}(f(t_1, t_2))$ with $\mathrm{rev}(\mathrm{FWD}) = \mathrm{BWD}$, $\mathrm{rev}(\mathrm{BWD}) = \mathrm{FWD}$, $\mathrm{rev}(\mathrm{EQ}) = \mathrm{EQ}$. A1 established baselines and found three things: a length-only classifier is highly consistent yet mostly wrong; fine-tuned encoders rarely confuse FWD with BWD; and their errors concentrate on whether a related pair is NEG.

This assignment proposes an approach built on those findings and designs the experiments to test it. We separate three questions: (i) does a stronger encoder help? (ii) does consistency-aware training help beyond the encoder? (iii) how much comes from exploiting the structure of the data rather than from better language understanding? The third question matters because a decoding rule can raise every metric without the model learning anything.

2 Justification

Evidence from A1. RoBERTa's per-class F1 was 0.839 (FWD), 0.839 (BWD), 0.725 (EQ) and 0.565 (NEG). Its largest error group (69 rows) was rejecting near-paraphrases as NEG. On reversible pairs, it confused FWD with BWD only twice in each direction. A1 reported 42 pairs in the category "NEG in one direction only"; on re-inspection that category also counted pairs predicted NEG in both directions. The strictly one-sided count is 15 for seed 42, and 15–19 per seed in our reruns. These errors cost both SoftCons and HardCons, and they cannot occur if both orientations are decided together.

Pair structure. The training set has 1,282 pairs present in both orientations ($2 \times 1{,}282 = 2{,}564$ rows) and 478 rows present in one orientation only, for 3,042 rows in total. In both train and dev, *every* row with a reversed partner is non-NEG and *every* row without one is NEG. Partners can be identified from `pair_id` in the participant file, which is available at test time; no gold label is needed. We treat this as an observed property of the released data, not a guarantee for the hidden test set. It also explains why most top leaderboard entries (9 of the other 25 English submissions on 8 November) report SoftCons of exactly 1.0: SoftCons fails a pair whenever either prediction is NEG, so a perfect score means no reversible pair was ever labelled NEG.

Encoder. DeBERTa-v3 (He et al., 2023) was the strongest model in the organisers' pilot. It was dropped in A1 because training produced NaN losses on Apple MPS. Training in fp32 on CUDA removes that obstacle.

Consistency training. Logical constraints have previously been added to NLI training as differentiable regularisers (Minervini and Riedel, 2018; Li et al., 2019). The reversal rule is such a constraint, so we test whether enforcing it during training helps.

3 Proposed Method

Figure 1 shows the pipeline. The model is a cross-encoder that reads "[CLS] t_1 [SEP] t_2" and outputs a distribution $p_{12} = \mathrm{softmax}(f_\theta(t_1, t_2))$ over the four labels.

Pair-aware batching. Batches are built from pairs,

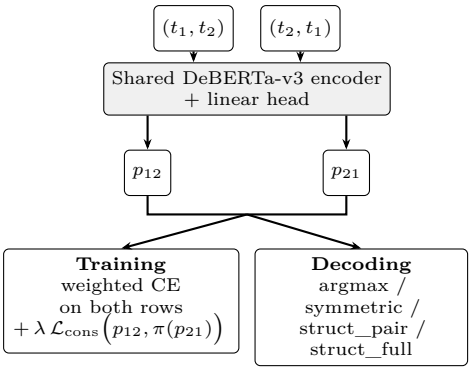


Figure 1: Proposed pipeline. Both orientations of a pair go through the same encoder in the same batch. π swaps the FWD and BWD probabilities. Rows without a partner (all NEG in train) receive only the cross-entropy term.

not rows: a reversible pair always contributes both orientations to the same batch, and a single row contributes itself. This makes the consistency term computable within a batch.

Training objective. Let π be the label permutation that maps a distribution for (t_2, t_1) onto the label space of (t_1, t_2): it keeps EQ and NEG and swaps FWD and BWD. For each pair in the batch,

$$\mathcal{L}_{\text{cons}} = \tfrac{1}{2}\Big[\mathrm{KL}\big(p_{12} \,\|\, \pi(p_{21})\big) + \mathrm{KL}\big(\pi(p_{21}) \,\|\, p_{12}\big)\Big], \quad (1)$$

and the total loss is

$$\mathcal{L} = \mathrm{CE}_w(y, p) + \lambda\, \overline{\mathcal{L}_{\text{cons}}}, \quad (2)$$

where CE_w is cross-entropy over all rows, with weight w_{NEG} on the NEG class and 1 elsewhere, and the bar denotes the mean over pairs in the batch. Single (NEG) rows have no partner, so they receive only the cross-entropy term.

Decoding rules. These change only how labels are chosen, so they apply to any trained model:

- **argmax**: $\hat{y} = \arg\max p_{12}$, as in A1.
- **symmetric**: for a row with a partner, $\tilde{p}_{12} = \frac{1}{2}\big(p_{12} + \pi(p_{21})\big)$, and its partner receives $\pi(\tilde{p}_{12})$. The two labels are then consistent by construction.
- **struct_pair**: symmetric, and rows with a partner may not be NEG.
- **struct_full**: struct_pair, and rows without a partner are labelled NEG.

The last two rules use the pair structure of Section 2. struct_full changes only rows without a partner, which never enter SoftCons or HardCons, so it can raise W-F1 but not HardCons relative to struct_pair.

4 Comparison Systems

We compare systems along two separate axes: what is trained, and how labels are decoded. Every trained system is evaluated under all four decoding rules.

- **A1 baselines**: majority class, length-only and feature-based logistic regression, BERT-base and RoBERTa-base (reported in A1).
- **M0**: RoBERTa-base retrained under the A2 protocol, so that it is directly comparable with the new systems.
- **M1**: DeBERTa-v3-base, cross-entropy only.
- **M2**: M1 + consistency loss ($\lambda = 1$).
- **M2b**: M2 with $\lambda = 0.5$, one seed, as a sensitivity check.
- **M3**: M2 + NEG weighting ($w_{\text{NEG}} = 2$).
- **Ensembles**: for each system, the three seeds' dev probabilities are averaged and decoded once.

The development leaderboard is used as external context only. Its top entries reach W-F1 above 0.95, which is likely to involve training on the public dev labels, so they are not a like-for-like comparison.

5 Evaluation Measures

We use the organisers' scorer unchanged. **W-F1**, the main score, averages per-class F1 weighted by class support over all 660 dev rows. **SoftCons** is the fraction of the 277 reversible dev pairs whose two predictions follow the reversal rule with neither being NEG; it ignores gold labels. **HardCons** is the fraction of those pairs with both orientations correct, so HardCons $\leq$ SoftCons. We also report Macro-F1 and per-class F1, because NEG is the smallest class (16%). As a diagnostic we count strictly one-sided NEG pairs.

Because SoftCons can be maximised without being correct (the length-only model in A1 scored 0.989), we judge every change on W-F1 and HardCons together, never on SoftCons alone.

6 Experimental Protocol

Splits. We hold out 10% of the training set as a validation split, grouped by `pair_id` so that a pair and its reverse never cross the split. Paired and single groups are sampled separately, preserving the label mix: 128 pairs and 48 singles (304 rows), leaving 2,738 training rows. The split uses a fixed seed (2027) and is identical for every run. Since no `pair_id` is shared between train and dev, there is no reversal leakage into dev.

Training. AdamW, learning rate 2×10^{-5}, weight decay 0.01, linear warm-up over 10% of steps, gradient-norm clipping at 1.0, 8 pairs (about 16 rows) per batch, maximum length 64 tokens, 6 epochs, fp32 throughout. Each configuration is trained with seeds 13, 42 and 87 on a Colab T4 GPU (about 2 minutes per RoBERTa run and 3.3 minutes per DeBERTa run).

Model selection. After each epoch we compute argmax W-F1 on the validation split, and keep the

best epoch. The dev set is never used for checkpoint selection or hyperparameter tuning.

Reporting. For each system we report the mean ± sample standard deviation over the three seeds, each scored separately. Ensembles are reported separately as single systems. A manifest in the repository maps every number to its configuration and prediction file, and all scores were recomputed from the saved probabilities (agreement with the original runs to within 10^{-16}).

Use of dev. Dev was not used to select checkpoints, but it was used to compare decoding rules and configurations, so the final configuration is informed by dev results. The dev scores below are therefore not an independent estimate of test performance.

7 Initial Implementation and Results

Implementation. The pipeline is implemented in PyTorch and Hugging Face Transformers. `src/train_a2.py` handles pair-aware batching, the loss and validation-based selection; `src/decoding.py` implements the four rules; `06_decoding_experiments.py` scores every model under every rule; and a Colab notebook runs the full set of experiments. All code and results are in the project repository.

Protocol check. M0 reproduces the A1 RoBERTa result (argmax 0.758 vs. 0.765 W-F1, within seed variation), so the new split and selection rule do not distort the comparison with A1.

Encoder. DeBERTa-v3 is the largest training-side gain (Table 1): +6.3 W-F1 and +8.0 HardCons under argmax, with seed standard deviations about four times smaller than RoBERTa's. Its gains are largest on EQ (+0.094 F1) and NEG (+0.104 F1), the two classes A1 identified as hardest (Table 2). With fp32 training no NaN losses occurred.

Symmetric decoding. Averaging each row with its partner's label-swapped prediction raises HardCons by 2–6 points for every system, with W-F1 unchanged or slightly higher, and removes all one-sided NEG pairs. It costs no training.

Structure-constrained decoding. This is the largest gain: about +10 W-F1 over argmax for every system. Per-class results show where it comes from: NEG F1 becomes 1.000 on dev, because the rule assigns NEG exactly to the rows without a partner. This gain reflects the construction of the dataset rather than better language understanding, and it transfers to the test set only if the test set is built the same way. Applied to the saved A1 models, the same rules raise RoBERTa from 0.765 to 0.874 W-F1 and BERT from 0.705 to 0.838, so the effect does not depend on the A2 models.

Consistency loss: a negative result. Under argmax, M2 has *lower* HardCons than M1 (0.827 vs. 0.839), and its one-sided NEG count falls from 9.0 to 6.7 per seed. Our interpretation is that the loss can be satisfied by predicting NEG in both orientations, which removes one-sided errors without making either prediction correct. Under the structural rules M2 is slightly ahead of M1, but the gap is within one standard deviation. After two seeds the gain looked clearer; the third seed reversed it (struct_full 0.915 for M2 vs. 0.927 for M1). The one-seed check with $\lambda = 0.5$ was no better (argmax 0.814, struct_full 0.921). We conclude that the consistency loss has no robust benefit in this setting. This is plausible given A1: the loss enforces directional agreement, while the encoders' remaining errors are mostly about NEG, not direction.

Neg weighting. M3 gives the best argmax W-F1, NEG F1 and Macro-F1, and the lowest variance under struct_full, but its margins over M2 are small and untested for significance. We report M3 as the best tested configuration, not as evidence that each of its components helps.

Ensembling. Averaging three seeds adds about one W-F1 point to every system and preserves their order (Table 3).

External check. We submitted the M3 struct_full ensemble, English only, to the Codabench development phase. It scored 0.936 W-F1, 1.0 SoftCons and 0.924 HardCons, identical to our local scorer, and ranked 7th of 26 English submissions at the time. This confirms the submission format and the local scorer. It is not an estimate of test performance, for the reason given under "Use of dev".

Reproducibility note. M1's struct_full W-F1 is identical to 16 decimal places for seeds 13 and 42. This is not a duplicated run: the seeds disagree on 24 dev rows, but each gets exactly 12 of them right, in mirrored FWD/BWD-versus-EQ swaps, so the confusion counts coincide. Symmetric decoding makes errors come in mirrored pairs, which makes such ties more likely.

8 Limitations and Plan for Assignment 3

Limitations. (1) The structural rules depend on a property seen in train and dev that may not hold for the test set; we will report results with and without them. (2) Dev informed the choice of configuration, so dev scores are optimistic. (3) Three seeds are few, and no significance tests have been run yet, so small differences between M1, M2 and M3 should not be interpreted. (4) Checkpoints are selected by argmax validation W-F1, although the best results use structured decoding; selection under the decoding rule might choose different epochs. (5) Only English is covered.

Planned experiments. Under the structural rules, every remaining error is an EQ/FWD/BWD choice on a paired row (about 21 of 277 pairs for the best ensemble). A3 will target that decision:

- **NLI-pretrained initialisation**: starting from a DeBERTa-v3 checkpoint already fine-tuned on MNLI-style data, which already models entailment

	argmax		symmetric		struct_pair		struct_full	
System	W-F1	HardCons	W-F1	HardCons	W-F1	HardCons	W-F1	HardCons
M0 RoBERTa	0.758 ±.026	0.759 ±.025	0.768 ±.034	0.821 ±.025	0.803 ±.036	0.869 ±.026	0.889 ±.022	0.869 ±.026
M1 DeBERTa-v3	0.821 ±.006	**0.839** ±.006	0.821 ±.006	**0.862** ±.015	0.853 ±.010	0.904 ±.008	0.919 ±.006	0.904 ±.008
M2 + cons.	0.820 ±.005	0.827 ±.011	0.822 ±.005	0.845 ±.006	0.870 ±.013	0.909 ±.008	0.923 ±.007	0.909 ±.008
M3 + NEG wt.	**0.827** ±.004	0.828 ±.015	**0.824** ±.004	0.842 ±.006	**0.877** ±.010	**0.911** ±.004	**0.925** ±.004	**0.911** ±.004

Table 1: Dev results, mean ± std over seeds 13, 42 and 87 (official scorer). SoftCons is 1.0 for both structural rules by construction. Under argmax it is 0.819, 0.892, 0.884 and 0.878 for M0–M3, and under symmetric 0.919, 0.931, 0.910 and 0.906. Bold marks the best mean in each column; most gaps between M1, M2 and M3 are within one standard deviation.

	Per-class F1 (argmax)				
System	EQ	FWD	BWD	NEG	Macro-F1
M0	0.719	0.864	0.823	0.516	0.730
M1	0.813	0.882	0.880	0.620	0.799
M2	0.801	0.879	0.883	0.633	0.799
M3	0.809	0.889	0.884	**0.643**	**0.806**

Table 2: Per-class F1 under argmax, mean of three seeds. Under struct_full, NEG F1 is 1.000 for every system by construction; EQ rises to 0.80–0.87 and FWD/BWD to 0.90–0.93.

	argmax		struct_full	
Ensemble	W-F1	Hard	W-F1	Hard
M0	0.784	0.794	0.912	0.895
M1	0.823	0.845	0.928	0.913
M2	0.830	0.845	0.930	0.917
M3	**0.832**	0.830	**0.936**	**0.924**

Table 3: Three-seed probability ensembles on dev (single systems, not means). The M3 struct_full ensemble was submitted to the Codabench development phase.

and paraphrase.

- **Three-class paired model**: training only on paired rows with EQ/FWD/BWD labels, since NEG is decided structurally; this also removes the "NEG both ways" solution that undermined the consistency loss.
- **Joint pair decoding**: choosing among the three valid joint labellings, (EQ,EQ), (FWD,BWD) and (BWD,FWD), by summed log-probability.
- **DeBERTa-v3-large** and **cross-backbone ensembles** (Lakshminarayanan et al., 2017).
- **Checkpoint selection under the final decoding rule**.
- **Significance testing**: paired bootstrap over dev pairs for W-F1 and HardCons, with more seeds where compute allows.
- **Error analysis** of the remaining wrong pairs, including how many have debatable gold labels.

A3 will also cover full ablations, comparison with related work, and the final paper-style report.

9 Conclusion

The two improvements that hold up are of different kinds. A stronger encoder genuinely improves classification (+6 W-F1, +8 HardCons) and is far more stable across seeds. Structure-constrained decoding gives a larger gain, but it relies on how the dataset was built, and we report it as such. Consistency-aware training, the most obvious way to target directional coherence, did not help, most likely because directional errors were already rare. The next step is the paired EQ/FWD/BWD decision, where every remaining error now lies.

References

DiCo-NLI Organizers. SemEval 2027 task 2: Directional-consistent fine-grained natural language inference (DiCo-NLI). `https://github.com/ilopezgazpio/SemEval-2027-Task-2-DiCo-NLI`, 2026. Accessed 4 October 2026.

Pengcheng He, Jianfeng Gao, and Weizhu Chen. DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing. In *The Eleventh International Conference on Learning Representations*, 2023.

Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Tao Li, Vivek Gupta, Maitrey Mehta, and Vivek Srikumar. A logic-driven framework for consistency of neural models. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pages 3924–3935, 2019.

Pasquale Minervini and Sebastian Riedel. Adversarially regularising neural NLI models to integrate logical background knowledge. In *Proceedings of the 22nd Conference on Computational Natural Language Learning*, pages 65–74, 2018.